

ADVANCED DOCKER CHEAT SHEET

Production Optimization, Security Hardening, Orchestration, and Deep Diagnostics

1. BUILD OPTIMIZATION & ADVANCED DOCKERFILE PATTERNS

Multi-Stage Build Pattern (Go/Rust/Node)

```
# Stage 1: Build environment
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-w -s" -o server .

# Stage 2: Minimal runtime environment
FROM scratch
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /app/server /server
USER 10001:10001
ENTRYPOINT ["/server"]
```

- **-ldflags="-w -s"** : Strips debug symbols (reduces size ~30%).
- **FROM scratch** : Zero-overhead baseline image.

Advanced BuildKit Features

Enable BuildKit using **export DOCKER_BUILDKIT=1**.

Mounting SSH Keys securely (no leakage):

```
RUN --mount=type=ssh ssh -T git@github.com
```

Persistent Compiler Cache (Speed up runs):

```
RUN --mount=type=cache,target=/root/.cache/go-build go build -o server .
```

Mounting Secrets safely at build-time:

```
RUN --mount=type=secret,id=api_key TOKEN=$(cat /run/secrets/api_key) && curl -H "Auth:$TOKEN" ...
```

2. ADVANCED NETWORK TOPOLOGIES & TROUBLESHOOTING

Custom Drivers & Network Creation

Overlay Network with Data Path Encryption (Swarm):

```
docker network create --driver overlay --opt encrypted --subnet 10.0.11.0/24 prod-overlay-net
```

Macvlan Network (Direct physical network bridge):

```
docker network create -d macvlan --subnet=192.168.1.0/24 --gateway=192.168.1.1 -o parent=eth0 mv-net
```

Network Debugging Sandbox Patterns

Attach a diagnostic container directly to another's namespace:

```
docker run -it --rm --network=container:target_container_name nicolaka/netshoot tcpdump -nnvv -i any
```

- Allows execution of **iptables**, **tcpdump**, **nslookup** inside target container network stack without contaminating runtime image.

3. PRODUCTION SECURITY HARDENING & ISOLATION

Kernel Capability Management

Never run containers with full root permissions. Drop all default privileges and explicitly whitelist required system calls:

```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE --cap-add=CHOWN nginx:alpine
```

Capability	Purpose
NET_BIND_SERVICE	Bind ports under 1024
SYS_TIME	Modify system clock
SYS_ADMIN	Dangerous broad root actions

Runtime & Storage Constraints

Read-Only Root Filesystem with Volatile App Paths:

```
docker run --read-only --tmpfs /tmp --tmpfs /run -v persistent-data:/app/uploads node:alpine
```

Prevent Privilege Escalation (Setuid binaries):

```
docker run --security-opt=no-new-privileges:true alpine
```

Custom SecComp Profile:

```
docker run --security-opt seccomp=custom_profile.json nginx
```

4. RESOURCE ALLOCATION & PERFORMANCE TUNING

Strict CPU & Kernel Scheduler Controls

Hard Limit (2.5 CPUs allocated via CFS scheduler):

```
docker run --cpus="2.5" nginx
```

Pin Container to Specific CPU Cores (NUMA performance):

```
docker run --cpuset-cpus="0,2" Redis:alpine
```

CPU Shares (Relative weight priority when CPU maxed):

```
docker run --cpu-shares=512 low-priority-job
```

Memory, Swap & OOM Management

Hard Limit + Swap Restrictions:

```
docker run -m 512m --memory-swap=1g java-app
```

- Allocates 512MB RAM and 512MB Swap. Set `--memory-swap` equal to `-m` to disable swap completely.

OOM Killer Weight Adjustment:

```
docker run --oom-score-adj=-1000 critical-db
```

- Protects critical production database containers from getting killed first by host kernel under RAM duress.

5. LOW-LEVEL DIAGNOSTICS, ENGINE TELEMETRY & AUDITING

Advanced Inspections & Runtime Filters

Extract Container IP Address via Go Template Format:

```
docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' container_id
```

Find Top 5 Heavy Containers by Real-Time Resource Usage:

```
docker stats --no-stream --format "table {{.Name}} {{.CPUPerc}} {{.MemUsage}}" | sort -k 2 -r | head -n 5
```

Investigate Structural Changes in Container Layers:

```
docker diff container_id
```

Engine Event Stream & Structural Auditing

Filter real-time engine activity stream for target failures:

```
docker events --filter 'event=die' --filter 'type=container'
```

Extract Exact Tail Logs with Precise Timestamp Match:

```
docker logs --since="2026-06-01T00:00:00" --until="30m" API_Gateway
```

Calculate Accurate On-Disk Storage Footprint Breakdown:

```
docker system df -v
```

6. PRODUCTION DOCKER COMPOSE CONFIGURATIONS

Production Specification (compose.yaml)

```
version: "3.8"
services:
  web:
    image: nginx:alpine
    deploy:
      replicas: 3
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
        reservations:
          cpus: '0.25'
          memory: 256M
      restart_policy:
        condition: on-failure
        delay: 5s
        max_attempts: 3
        window: 120s
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

7. EMERGENCY RECOVERY & SYSTEM SANITIZATION

Aggressive Nuclear Clean (Removes all dangling/unused states, stop containers first):

```
docker system prune -a --volumes --force
```

Identify and Extract Corrupted / Broken Graph Driver Paths:

```
docker inspect --format '{{.Id}}: {{.GraphDriver.Data.WorkDir}}' $(docker ps -a -q)
```